Extrapolating Neural Calculator: Generalisation Beyond the Training Distribution

Machine Learning 2025/26 — Final Project (Full Practical Track)
Javier Vaca, Gabriele Toro, Marcos García

Abstract

We learn a *neural calculator* that maps an arithmetic expression composed of single digits, the operators + and −, and parentheses to its integer value, without any symbolic evaluation. We frame the task as scalar regression on a character sequence and compare two substantially different architectures: a unidirectional LSTM with last-token read-out and a Transformer encoder. The central challenge is extrapolation to longer and more deeply nested expressions than those seen in training. Both models clear all required thresholds, with the LSTM achieving the best performance, reaching mean exact-match accuracies of 99.106% / 95.506% / 83.770% across multiple runs on the ID / OOD / long-sequence test sets. A per-axis error analysis reveals that the two architectures fail for *different* reasons: the LSTM is limited by the magnitude of the result, while the Transformer is limited by sequence length and nesting depth. The LSTM generalises consistently better under distribution shift, suggesting that recurrent inductive biases remain better for this symbolic reasoning task.

1 Problem and approach

Neural networks are often used in many sequence tasks. In this project, the input sequence is a string over the alphabet `[0-9 + - ( )]` in which every operand is a single digit; the target is the signed integer value of the expression, which in the provided data lies in $[-99, 99]$. Exact-match accuracy rounds the prediction to the nearest integer. Unlike traditional calculators, the model is not allowed to perform symbolic evaluation or use any hand-crafted arithmetic rules. Instead, it must learn the underlying computation exclusively from training examples.

The primary focus is generalisation under distribution shift. Training and validation expressions are relatively short and structurally limited, whereas the evaluation includes an out-of-distribution (OOD) set with different structural properties and a long-sequence set with substantially longer formulas. To study how architecture affects generalisation, we compare two neural approaches: a unidirectional Long Short-Term Memory (LSTM) network that predicts from its final hidden state, and a Transformer encoder. Both are trained as regression systems that directly predict the numerical result. We report exact-match accuracy, MAE and RMSE on all four splits, and analyse how errors vary with sequence length, parenthesis depth, operator count, and target magnitude.

2 Models

We train two architecturally distinct sequence models that map a tokenized expression to a single real-valued prediction. Both models share a character vocabulary (15 symbols, index 0 reserved for padding) and a learned embedding with `padding_idx=0`; padding is masked in both so it never contributes to pooling or attention.

Model A — LSTM with last-state read-out. A single-layer unidirectional LSTM processes the embedded sequence left to right, and the prediction is produced from the hidden state *at the last token* through a small feed-forward head. The key design choice is that padding is added at the *front* of the sequence (pre-padding), so the last position always coincides with the last real character of the expression. This exploits a structural property of the task: since arithmetic accumulates left to right, the hidden state after the final character already encodes the running result, making an explicit pooling over all tokens unnecessary.

Motivation. LSTMs have a strong inductive bias towards sequential, left-to-right processing and carry state across the whole sequence, which matches the way the value of an expression is accumulated. We found a single unidirectional layer with last-token read-out both simpler and stronger than a bidirectional attention-pooled variant: the recurrent state is exactly where the partial computation lives, so the dominant hyperparameter is the hidden size.

Hyperparameters. Embedding dimension $d_e = 16$, hidden dimension $d_h = 256$, 1 layer, dropout $p = 0.1$, unidirectional. These hyperparameters were selected through manual tuning, choosing the configuration that yielded the best validation performance.

Development History. Before arriving at the final architecture, two alternative models were explored and discarded. A Bidirectional LSTM with attention pooling and stacked layers was the first candidate: while it captured some sequential structure, its performance on the OOD and long-sequence splits was insufficient, suggesting limited ability to track nested parentheses across positions. A GRU-based model was then tested as a lighter alternative, offering faster training but showing similar extrapolation weaknesses. Both models were outperformed by the final approach across all test splits.

Model B — Transformer. The second model is a Transformer encoder based on self-attention. Input tokens are embedded and augmented with sinusoidal positional encodings, which provide information about token order since self-attention alone is permutation-invariant.

The model is composed of multiple stacked encoder layers, where each layer allows every token to directly interact with all others through self-attention. This enables efficient modeling of long-range dependencies and non-local relationships, which are important in expressions with nested parentheses.

To obtain a fixed-size representation, we apply mean pooling over the output token representations, excluding padding positions. The resulting vector is passed through a feed-forward network that produces the final scalar prediction.

Motivation. Transformers process all tokens in parallel through self-attention, allowing every token to directly interact with every other token regardless of distance. In theory, this is beneficial for arithmetic expressions because the value of a sub-expression depends on non-local structure. Besides, we used sinusoidal positional encodings because they are defined for arbitrarily long sequences without retraining, unlike learned positional embeddings.

Hyperparameters. 4 encoder layers, 4 attention heads, model dimension 128, feed-forward dimension 384, dropout 0.1. We also arrived at these specific values through a process of manual trial and error to determine the best-performing setup.

Development History. Before settling on mean pooling, we tried three other ways of turning the sequence of token representations into a single vector. The first added a special token at the end of the expression, meant to collect the final result; in practice the model could not reliably concentrate all the needed information into that one position, especially for longer expressions. The second used only the representation of the last token as the summary of the whole sequence, but this relied too much on the final characters rather than the expression as a whole, which hurt out-of-distribution performance. The third kept, for each dimension, the largest value across all tokens (max pooling), but this lost the positional information needed to resolve nested parentheses. Mean pooling outperformed all three on the validation, OOD, and long-sequence splits.

3 Training

Both models are trained in a supervised regression setting. We use the training split provided by the dataset (12,000 examples) and monitor performance on the validation set (2,000 examples).

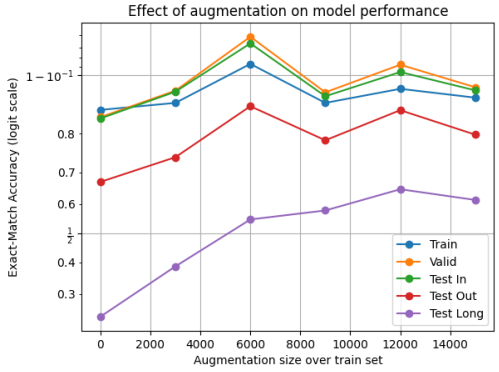


Figure 1: Exact-match accuracy as a function of the number of synthetic examples added to the training set. Results are shown for all evaluation splits.

Data augmentation. To improve coverage of deeply nested and structurally diverse expressions, we initially extended given training examples by appending extra $\pm d$ terms within the remaining character budget. However, this strategy performed poor results, and led us to generated synthetic examples directly from the project grammar. Two types of expressions were created: *deep chains*, which increase nesting depth, and *broad expressions*, which introduce greater structural variety. All generated examples satisfy the original length and value constraints. Together, these strategies increased coverage of nesting depth. An augmentation size of 6,000 examples was selected, since larger augmentation sets did not improve performance (Fig. 1). The Transformer is trained on this augmented set; the LSTM, which already generalised near-perfectly, is trained on the original data without augmentation.

Optimization. Training is performed using the AdamW optimizer with learning rate 5×10^{-4} and weight decay 10^{-2}. The loss function is Smooth L1 loss, which provides a balance between mean absolute error and mean squared error, offering robustness to outliers in target values.

Training procedure. Both models use learning rate 5×10^{-4}, mini-batches of size 32 with shuffling. Each training epoch consists of a full pass over the dataset, where gradients are computed via backpropagation and updated using the optimizer. Gradient clipping with a maximum norm of 1.0 is applied to stabilize training and prevent exploding gradients. Both models are implemented in PyTorch with masked computation for padding.

Learning rate scheduling. We use a learning rate scheduler to improve convergence. For LSTM, a cosine annealing schedule is applied over the full training horizon. For Transformer, we additionally support an optional warm-up phase using a OneCycleLR schedule, which gradually increases the learning rate at the beginning of training before annealing.

Evaluation during training. We have fixed 50 epochs. After each epoch, performance is evaluated on the validation set without gradient computation. Training and validation losses are logged to monitor convergence and detect overfitting.

4 Results

Training a network from a random initialisation is stochastic, so single-run figures can be misleading. We therefore trained each architecture over multiple independent runs (with different random seeds) and report two complementary views. Tables 1 and 3 give the metrics of the single *best run*, which characterise the attainable performance of each model. Tables 2 and 4 give the *mean exact-match accuracy* across all runs with its 95% confidence interval, which characterise expected performance and stability. No run was discarded: every run enters the average, so the reported means are unbiased estimates. Across-run statistics were computed for exact-match accuracy only; MAE/MSE/RMSE are reported for the best run.

Table 1: Unidirectional LSTM — best run.

Dataset	EM (%)	MAE	MSE	RMSE
Train	99.94	0.035639	0.003041	0.055148
Valid	99.60	0.038456	0.004972	0.070511
Test In	99.85	0.038805	0.003937	0.062742
Test Out	99.20	0.060915	0.018700	0.136748
Test Long	98.07	0.107750	0.148749	0.385679

Table 2: Unidirectional LSTM — mean exact-match accuracy over multiple runs.

Dataset	Mean (%)	95% CI
Train	99.86	[99.80, 99.93]
Validation	99.12	[98.77, 99.46]
Test In	99.10	[98.63, 99.57]
Test Out	95.50	[93.38, 97.62]
Test Long	79.60	[69.12, 90.17]

Table 3: Transformer encoder — best run.

Dataset	EM (%)	MAE	MSE	RMSE
Train + Aug	93.37	0.227134	0.615240	0.784372
Valid	94.65	0.189258	0.273250	0.522733
Test In	95.25	0.177311	0.150541	0.387996
Test Out	88.10	0.343655	1.222605	1.105715
Test Long	61.80	1.342894	11.145906	3.338549

Discussion. The LSTM dominates the Transformer on every split, and the gap widens precisely where it matters most: on the long-sequence test the LSTM averages 83.8% against the Transformer's 53.8%. The confidence intervals expose a second-order effect. In distribution the LSTM is extremely stable (a 95% CI within roughly ± 0.5 points on Train, Validation and Test In), but its variability grows sharply under extrapolation, the long-sequence interval spanning about ± 6.4 points; the same pattern, amplified, holds for the Transformer. The instability between different training runs is mainly concentrated in the most difficult tasks, where the model has to predict entirely new data. This explains the significant gap between the LSTM model's peak performance (98.07%) and its average performance (83.77%). To get a realistic and reliable measure of the model's ability to handle new situations, we should trust the average score and its confidence interval, rather than the single best result.

Table 4: Transformer encoder — mean exact-match accuracy over multiple runs.

Dataset	Mean (%)	95% CI
Train	88.82	[86.54, 91.09]
Validation	90.39	[88.21, 92.56]
Test In	90.16	[87.82, 92.44]
Test Out	80.91	[78.02, 83.80]
Test Long	53.80	[50.29, 57.31]

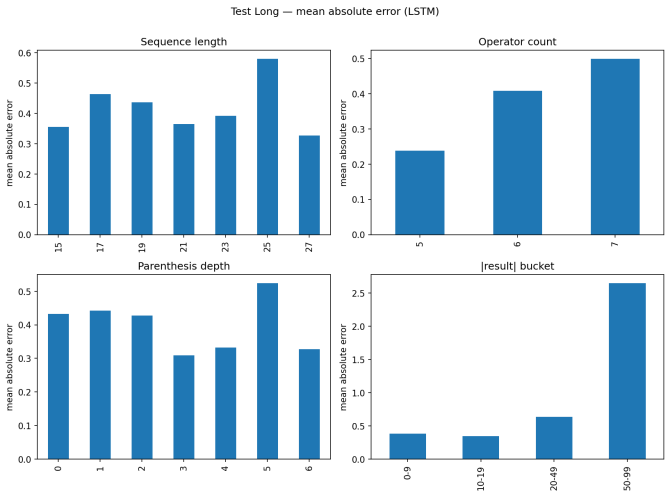
5 Extrapolation analysis

For each model we computed the per-example absolute error on the long-sequence split and grouped it along these four axes (Fig. 2). The two architectures fail along *different* axes, which is the central finding of this analysis.

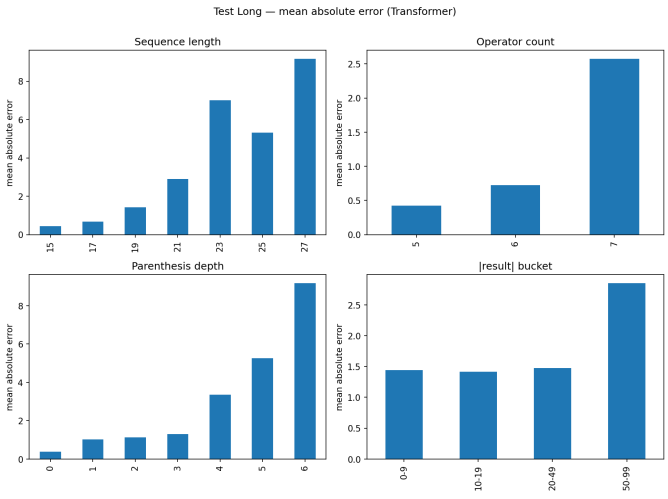
The LSTM is limited by result magnitude. For the recurrent model (Fig. 2, up) the error is essentially flat across sequence length (MAE $\approx$ 0.3–0.6), operator count (a mild rise from 0.24 to 0.50) and parenthesis depth ($\approx$ 0.3–0.5, with no upward trend). The single dominant effect is the magnitude of the target value: MAE stays near 0.4–0.6 for results up to 49 but jumps to about 2.7 for results in $[50, 99]$. The LSTM has learned to combine a bounded number of single-digit terms accurately regardless of how the expression is structured, but its accuracy collapses once the *value* of the result leaves the range densely covered in training, where targets cluster near zero. Its limitation is therefore *numeric*.

The Transformer is limited by structural size. The encoder (Fig. 2, right) shows the opposite profile. Its error grows sharply and monotonically with sequence length (from ≈ 0.4 at length 15 to ≈ 9 at length 27), with operator count (from 0.4 to 2.6) and with parenthesis depth (from 0.4 to $\approx$ 9), while result magnitude has only a weak effect (roughly flat around 1.5, rising to 2.8 only in the largest bucket). Self-attention copes with large values but degrades as the sequence becomes longer and more deeply nested than anything seen in training — the classic difficulty of extrapolating positional structure beyond the training range. Its limitation is therefore *structural/positional*.

Interpretation. The contrast is informative. The recurrent model processes the expression step by step and accumulates structure robustly, so its weakness is the numeric output range rather than the shape of the input; the Transformer handles values comfortably but is sensitive to growth in length and nesting depth. This also explains why the LSTM generalises better overall on the long split: large-magnitude results are comparatively rare, whereas longer and deeper expressions are exactly what the long split is designed to contain, and that is the regime the Transformer handles worst. Augmentation within the 21-character budget mitigates both effects only partially: it cannot manufacture many large-magnitude targets for the LSTM, nor enough very long, deeply nested expressions for the Transformer.



(a) LSTM — limited by result magnitude.



(b) Transformer — limited by sequence length and depth.

Figure 2: Mean absolute error on the long-sequence split, grouped by sequence length, operator count, parenthesis depth, and result magnitude. The two models fail along different axes: the LSTM degrades almost only with result magnitude (a numeric limit), whereas the Transformer degrades with sequence length, operator count and nesting depth (a structural/positional limit).

6 Conclusions

In this project, we framed arithmetic evaluation as a character-level sequence-regression task, comparing a unidirectional LSTM against a Transformer encoder. Both models achieved high in-distribution accuracy and demonstrated significant extrapolation capabilities. While the Transformer required synthetic data augmentation to handle structural variations within the training budget, the LSTM generalised well using only the original data, ultimately achieving superior performance, especially on the long-sequence split.

Our extrapolation analysis highlights a fundamental difference in how these architectures fail. The LSTM is constrained *numerically*: its accuracy drops when the target value falls outside the densely sampled training range, yet it remains highly robust to sequence length and parenthesis nesting. Conversely, the Transformer is constrained *structurally*: it handles large numbers well but struggles as expressions become longer and more deeply nested than those seen during training. Since the long-sequence test set primarily introduces structural complexity rather than large numerical values, the LSTM naturally outperforms the Transformer in this regime.

Depending on the model, the recommended next steps within the project guidelines are different.For the LSTM, we need to improve how the model handles very large values through targeted data augmentation and better output structuring. For the Transformer, we should improve its positional encodings for longer sequences and train it progressively, gradually increasing the length and complexity of the data.

Contributions. *Marcos García* worked on the dataset pipeline and contributed to the implementation of the training infrastructure and to the writing and organization of the report. *Javier Vaca* focused on the design of both models, as well as the overall training procedure. *Gabriele Toro* carried out the evaluation, including the extrapolation analysis and experimental comparison, and contributed to the writing and organization of the report. All members participated in discussions, interpretation of results, and final revisions.

References

[1] S. Hochreiter and J. Schmidhuber. Long Short-Term Memory. *Neural Computation*, 1997.

[2] A. Vaswani et al. Attention Is All You Need. *NeurIPS*, 2017.

[3] A. Trask et al. Neural Arithmetic Logic Units. *NeurIPS*, 2018.

[4] Anthropic. Claude (used for data augmentation code and text refinement). 2026.